

Day-22

Array List

↳ dynamic

import
Jav, JAL (*)

ArrayList

Array → Static

primitive data type

int arr[] = new int[10]

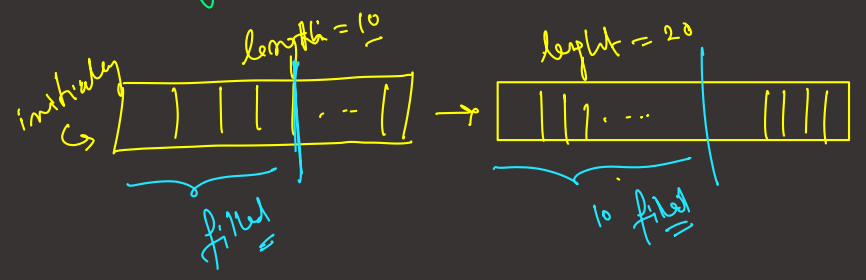
length = 10.



non-primitive

ArrayList<Integer> list = new ArrayList<>();

- Float
- Character
- String
- Boolean
- Double



Sorting

Collections.sort(list);

Reverse Order Sorting

Collections.sort(list, Collections.reverseOrder());

non-primitive data type

Note for normal array.

Arrays.sort(arr);

But for reverse order

Arrays.sort(arr, Collections.reverseOrder());

Integer arr[] = new Integer(5);

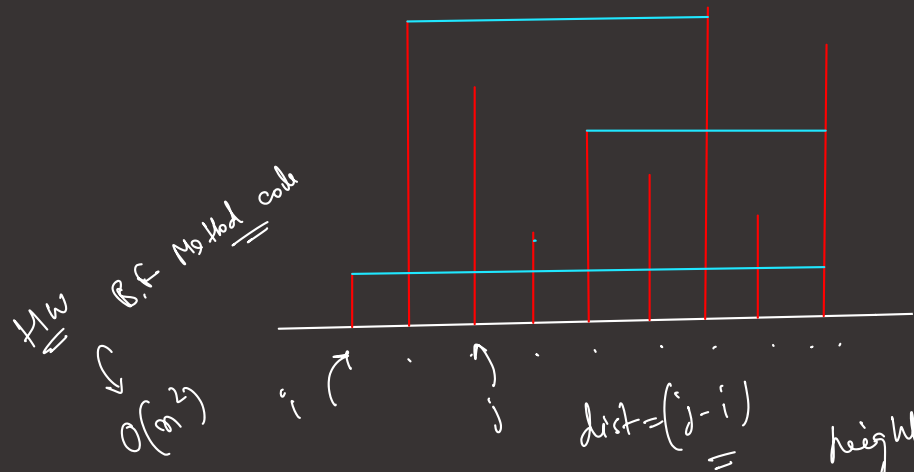
Operation

- list.add()
- get()
- remove()
- set()
- contains()

Q Container with most water

for given n lines on x -axis, use 2-line to form a container such that it holds the maximum water.

height = [1, 8, 6, 2, 5, 4, 8, 3, 7]



$i = 0$ to $(n-2)$
 $j = i+1$ to $(n-1)$

Initial water = $-\infty$

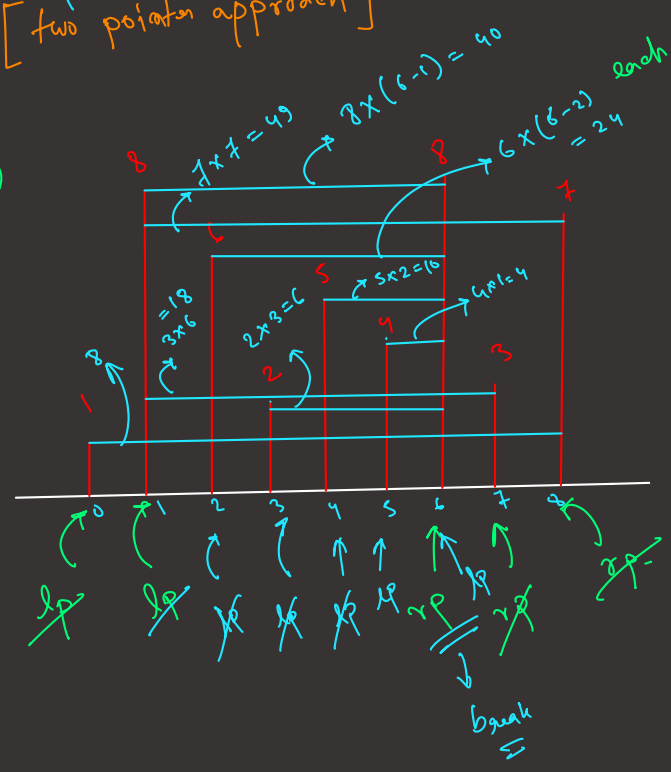
currWater = height * dist

after each iteration,
water = $\max(water, currWater)$

Q2 Approach 2

Optimized Code [two pointer approach] $O(n)$

while (lp != rp)



currWater = $-\infty$

each time
width = $rp - lp$
height = $\min(arr[lp], arr[rp])$
water = width * height.
max water optimization.
if ($arr[lp] \leq arr[rp]$)
 $lp++$
else
 $rp--$

